

DynamoDB

Earth movers are epic pieces of machinery, able to shuffle around massive bits of dirt and other materials with great ease. DynamoDB is a bit like the rented earth mover of NoSQL databases. You don't have to build it yourself or fix it when it's broken; you just have to drive it and pay for your usage. But it's complex to handle so you'll need to make very intelligent decisions about how to use it lest you end up incurring unexpected costs or jamming the engine.

DynamoDB is a cloud-based database available through Amazon Web Services (AWS), the cloud computing division of e-commerce giant Amazon (the same Amazon from which you may have purchased this book). You may know AWS as the creator of a dizzying, ever-expanding array of widely used cloud services, from the Simple Storage Service (S3) to the Elastic Compute Cloud (EC2) and far beyond (there could be over 100 services by the time you read this).

Despite the emergence of serious competitors, such as Microsoft Azure and Google Cloud Platform, AWS remains the leader of the Infrastructure-as-a-Service (IaaS) paradigm that has brought cloud computing to the masses, and DynamoDB is AWS's most significant contribution—thus far—to the world of NoSQL. The cloud has opened vast new horizons for everyone from lone-wolf developers to Fortune 500 companies, and any book on the NoSQL paradigm would be incomplete without this pioneering cloud database.

DynamoDB: The “Big Easy” of NoSQL

Six out of the seven databases that we cover in this book are easy enough to run on your laptop. But running systems such as HBase, CouchDB, and others in production—and using them for applications handling massive workloads—is a much different matter. Even databases that are a famously easy to operate at smaller scale, such as Redis, present major challenges in

production environments, usually requiring skilled (and expensive!) admins and operations specialists on hand.

DynamoDB is a different story—and an outlier in this book. You don’t have to install it, start it, or maintain it. You can sign up for an AWS account, create a DynamoDB table, and just go. As you’ll see, DynamoDB does require *some* operations-style thinking and preparation, but you’ll never need to provide it an XML configuration à la HBase or set up a complex cluster à la Mongo. DynamoDB is a database that runs itself and yet is capable of fulfilling some of your most ambitious “webscale” dreams, offering consistently fast performance no matter how much data you’re storing.

So just how “webscale” are we talking here? Some facts:

- You can store as many items as you want in any DynamoDB table (more on tables and items later).
- Each item (the equivalent of a row in an SQL database) can hold as many attributes as you want, although there is a hard size limit of 400 KB per item (that limit will likely grow in the future).
- If you get data modeling right, which will occupy a decent chunk of this chapter, you should experience very little performance degradation even when your tables store petabytes of data.
- Over 100,000 AWS customers currently use DynamoDB.
- DynamoDB handles well over a trillion total requests a day (across all AWS customers).

But DynamoDB isn’t interesting just because it’s big and cloud-based and managed by experts you don’t have to hire and fire. It’s also a system very much worth learning in its own right, providing a familiar yet unique data model and an array of features you won’t find in any of the other databases in this book. While some folks may be reticent to trust a cloud database that’s managed by someone else, a variety of forward-thinking tech companies, including Airbnb, Adobe, Siemens, and Comcast, have taken the plunge and use DynamoDB as one of the core databases driving their platforms.

DynamoDB and the (Almost) Ops-Free Lifestyle

If you’re running a database yourself or as part of a team, you can expect many of the following to keep you awake at night: ensuring speedy, predictable performance; handling unforeseeable hardware outages and network failures; scaling out disk capacity to meet unexpected spikes in demand; and enabling application developers to quickly get up and running using your database.

On top of that, DBAs with a background in NoSQL aren’t exactly a dime a dozen, so hiring, training, and scaling out a team of DBAs for Mongo, HBase, or another NoSQL database is nothing to sneeze at. DynamoDB doesn’t completely rid your life of these kinds of issues, but if you use it right it can take an enormous bite out of them.

There are also a number of secondary reasons why you might want to consider DynamoDB:

- You can use it in any of AWS’s many datacenters across the entire globe. As of July 2017, AWS offers DynamoDB in forty-two Availability Zones (AZs) in sixteen geographic regions, with plans to expand into at least eight more AZs in three additional regions.
- All data in DynamoDB is stored on high-performing Solid State Disks (SSDs) and automatically replicated across multiple availability zones within an AWS region (which guarantees redundancy even within a single region).
- You can expect genuine downtime out of DynamoDB only in the rare event that an entire AWS datacenter goes down.

Datacenter outages are the Achilles heel of the cloud, and a very real risk that you should keep in mind. We’ve all experienced Netflix, Instagram, and other widely used services going down for hours at a time due to outages in Amazon’s massive us-east-1 datacenter in Northern Virginia. AWS and DynamoDB aren’t perfect, but their track record is exceedingly good, if not downright pristine. Using a database like DynamoDB won’t grant you a *completely* ops-free lifestyle, but it may just enable you to refocus a huge chunk of your attention and resources onto other things, and for that reason alone it’s worth a look.

The Core Storage Concepts: Tables, Items, and More

DynamoDB’s data model is a bit tricky to define using standard “NoSQL” categories. It strongly resembles the data model of a key-value store such as Redis in that it wasn’t really built to provide the rich queryability of an RDBMS such as Postgres. Although DynamoDB *does* have some interesting querying features, which we’ll learn about shortly, it really soars when you know what you’re looking for in advance, which is a hallmark of key-value stores. If you’re building an application that uses DynamoDB, you should always strive to architect it so that your data is associated with certain “natural” keys that allow for easy discoverability—for example, the ability to find user data on the basis of unique usernames.

Not to Be Confused with Dynamo

Technologically, DynamoDB originally drew heavily on concepts derived from a distributed, eventually consistent storage system called *Dynamo* created to address Amazon's own data storage problems (and massive ones at that). Amazon's theoretical research into the distributed database domain resulted in the so-called "Dynamo paper" (actually titled *Dynamo: Amazon's Highly Available Key-value Store*),^a which exerted a seminal influence on widely used NoSQL databases such as Riak, Cassandra, and Voldemort.

It's unclear how faithful DynamoDB is to the concepts in the Dynamo paper, as Amazon keeps most under-the-hood implementation details under wraps, but the paper itself is a treasure trove of rich theoretical explorations of distributed database concepts. Throughout this book, we'll be careful to always use the term *DynamoDB* to distinguish the public-facing AWS service from the internal Dynamo and its associated paper.

a. <http://dl.acm.org/citation.cfm?id=1294281>

There are aspects of DynamoDB's data model, however, that are reminiscent of RDBMSs such as Postgres. The first point of overlap is that all data in DynamoDB is stored in *tables* that you have to create and define in advance, though tables have some flexible elements and can be modified later. You can create, modify, and delete DynamoDB tables at will using an interface called the *control plane*. If you're used to interfaces like Postgres's *psql*, which we explored in [Chapter 2, PostgreSQL, on page 9](#), then the control plane should be familiar to you.

The second point of overlap is that you store *items* inside of tables. Items roughly correspond to *rows* in RDBMSs; they consist of one or more *attributes*, which roughly correspond to RDBMS columns. Earlier in the book, you learned about databases such as Mongo and Couch that have no concept whatsoever of predefined tables. DynamoDB requires you to define only *some* aspects of tables, most importantly the structure of keys and local secondary indexes, while retaining a schemaless flavor.

The last point of overlap with RDBMSs is that DynamoDB enables you to query data based on *secondary indexes* rather than solely on the basis of a primary key (think back to secondary indexes in Postgres). This means that you can perform queries in DynamoDB that are essentially equivalent to SQL queries like these:

```
/* Remember: you can't actually use SQL syntax with DynamoDB;
these examples are just for show */
SELECT * FROM chevys WHERE make = "nova";
SELECT * FROM pro_sports_teams WHERE city = "cleveland";
SELECT * FROM presidents WHERE first_name = "Jethro"; /* OOPS! None found! */
```

You can even perform *range queries*:

```
SELECT * FROM pearl_jam_albums WHERE
  title <= "Ten";
SELECT * FROM john_cusack_films WHERE
  title BETWEEN "Better Off Dead" AND "High Fidelity";
SELECT * FROM oscar_wilde_quotes WHERE
  quote LIKE 'I have nothing to declare%';
```

DynamoDB’s Consistency Model

Now that we have a basic outline of DynamoDB’s “key-value plus” data model, a question naturally emerges: How does DynamoDB fit in with the so-called CAP theorem that we discussed in the last chapter? Are we dealing with an eventually consistent database that may turn up stale data from time to time (such as CouchDB and others in the NoSQL landscape)? Or are we dealing with a strongly consistent, ACID-compliant, transactional database that only ever returns the most up-to-date value that we’re seeking?

The answer: Yes, please! Everybody gets a car! DynamoDB actually supports *both* consistency models. Even better, you can specify which consistency model you want on a *per-read basis*.

So when you query DynamoDB, your application can say either...

- I want the most up-to-date value, even if it costs me some extra latency or, heaven forbid, the value isn’t currently available at all, *or*
- I’ve got a tight schedule, so give me what you’ve got right now, even if it’s a bit stale.

Always bear in mind, however, that “stale” in the universe of DynamoDB doesn’t mean hours; it probably means milliseconds, and the trade-off may be acceptable in plenty of cases (make sure to run it by your CTO, though). The flexibility, however, is nice, and the ability to query the *same data* using both models can really come in handy.

The downside of strongly consistent reads, as in other systems, is that they may not be available in case of network, hardware, or other outages. Death, taxes, and the CAP theorem: there’s no escaping them. The only real “solution” is to use strong consistency only when truly necessary and to design your application to be prepared to deal with an unresponsive database (rare as it may be with DynamoDB). Consistent reads also “cost” twice as much in terms of read capacity than non-consistent reads.

Another important thing to note about consistency is that DynamoDB *supports only item-level consistency*, which is analogous to row-level consistency in RDBMSs. There are *no* atomic operations across items, which means no consistency for batch operations. And when you run queries against indexes or whole tables, do not *ever* expect that the result set will be 100 percent up-to-date. Item-level consistency is a good thing to have, but if consistency across items is a necessity for your use case, you should explore other databases.

As you can see, these kinds of querying capabilities take DynamoDB beyond what you'd find in a more straightforward key-value store (like the one you'll see in the next chapter, on Redis). So we'll call DynamoDB's data model *key-value plus* for short to account for these borrowings from the relational paradigm.

In spite of these SQL-flavored capabilities, though, there are firm limits to the DynamoDB/RDBMS parallels. Most importantly, if you need querying capabilities that go beyond the simple ones in the previous example, you'll have to implement them on the application side, or just use a different database (or use other cloud services in conjunction with DynamoDB, as we'll do on Day 3). Furthermore, DynamoDB has no concept of things like joins between tables; the table is the highest level at which data can be grouped and manipulated, and any join-style capabilities that you need will have to be implemented on the application side, which has its own downsides.

So that provides a little bit of background, historical and technological, for DynamoDB. It's time to dig much deeper using real interactions with the database.

Day 1: Let's Go Shopping!

Fun fact: Dynamo—the database that inspired the later DynamoDB—was originally built with the very specific purpose of serving as the storage system for Amazon's famous shopping cart. When you're building a shopping cart application, the absolute, unbreakable categorical imperative guiding your database should be this: *do not lose data under any circumstances*. Losing shopping cart data means losing money directly. If a user puts a \$2,500 mattress in their Amazon shopping cart and the database suddenly forgets that, then that's money potentially lost, especially if the data is lost just before checkout.

Multiply a mistake like that times thousands or even millions and you get a clear sense of why Amazon needed to build a fault-tolerant, highly available database that never loses data. The good news is that you get to reap the benefits of Amazon's efforts and use DynamoDB to your own ends in your own applications.

As a first practical taste of DynamoDB, we'll set up a DynamoDB table that could act as the database for a simple shopping cart application. This table will be able to store shopping cart items (it looks like Amazon's use of the "item" terminology was no accident). We'll perform basic CRUD operations against this table via the command line.

But first, a stern warning: *DynamoDB is not free!* It is a paid service. AWS offers a free tier for virtually all of its services so that you can kick the tires without immediately incurring costs, but make sure to check the pricing guide¹ before you start going through the examples in this book. We won't make super intensive use of AWS services, so it should cost you at most a few dollars, but due diligence may just save you from an unpleasant end-of-the-month surprise.

Before you can get started with this section, you'll need to create an AWS account for yourself using the AWS console.² That part is pretty self-explanatory. Don't worry about familiarizing yourself with the AWS console for now, as most of the examples in this chapter will use the command line. Once you've signed up with AWS and can access your user console (which shows you all of AWS's currently available services on the landing page), download the official AWS CLI tool using pip, the Python package manager:

```
$ sudo pip install aws
```

If you run `aws --version` and get a version string like `aws-cli/1.11.51 Python/2.7.10 Darwin/16.1.0 boto3/1.5.14`, you should be ready to go. All of the `aws` tool's commands are of the form `aws [service] [command]`, where `service` can be `dynamodb`, `s3`, `ec2`, and so on. To see the commands and options available for DynamoDB specifically, run `aws dynamodb help`.

Once the CLI tool is installed, you'll need to configure it by running the following:

```
$ aws configure
```

Running DynamoDB Locally

If you ever decide to develop a full-fledged application of your own using DynamoDB, you might want to check out DynamoDB Local,^a which is a version of DynamoDB that you can run on your own machine that's fully API compatible with hosted DynamoDB. It's available as a Java JAR and works on Windows, Mac OS, and Linux.

We won't use DynamoDB Local for the exercises here because we'll need to use the hosted version in conjunction with other AWS services, but it's a very nice thing to have in your development toolbelt.

a. <https://aws.amazon.com/blogs/aws/dynamodb-local-for-desktop-development>

1. <https://aws.amazon.com/pricing>
2. <https://console.aws.amazon.com>

That will prompt you to input the AWS access key ID and secret access key for your account and enable you to choose a default AWS region when using the tool (us-east-1 is the default) and output format for CLI commands (json is the default).

The DynamoDB *control plane* is the set of commands used to manage tables. Our first action using the control plane will be to see which tables are associated with our account:

```
$ aws dynamodb list-tables
{
  "TableNames": []
}
```

As expected, there are no tables associated with our account, so let's make a very simple shopping cart table in which each item has just one attribute: an item name stored as a string.

```
$ aws dynamodb create-table \
  --table-name ShoppingCart \
  --attribute-definitions AttributeName=ItemName,AttributeType=S \
  --key-schema AttributeName=ItemName,KeyType=HASH \
  --provisioned-throughput ReadCapacityUnits=1,WriteCapacityUnits=1
```

DynamoDB's *Almost* Schemaless Data Model

In the create-table operation, it *seems* like the ShoppingCart table is being created with a strict schema according to which items in the table can have only an ItemName attribute. But DynamoDB doesn't work that way. Whenever you create a table, you only have to define attributes *that function as keys* (sometimes referred to as *key attributes*).

So we could store items in our shopping cart table that have any number of other properties (brand name, year manufactured, ISBN, whatever) if we wanted to, without having to specify those attributes when we create the table. The only restriction on our ShoppingCart table is that each item must have an ItemName.

But there's a catch here: although schema restraints apply only to key attributes, you can't query for attributes that aren't specified as keys or indexes when you create the table (more on indexes later). So if you started storing items with a brand name attribute in the ShoppingCart table, you wouldn't be able to discover items by brand name. If you wanted to do that, you'd have to create a new table and add the brand name as a key or index. And so even though schema design doesn't force you into a straitjacket, you should make that decision very carefully.

This is in contrast to a database like Mongo, which is schemaless but allows you to query for whatever fields you want at any time.

The output of that command should be a JSON object describing our newly created table:

```
{
  "TableDescription": {
    "TableArn": "arn:aws:dynamodb:...:table/ShoppingCart",
    "AttributeDefinitions": [
      {
        "AttributeName": "ItemName",
        "AttributeType": "S"
      }
    ],
    "ProvisionedThroughput": {
      "NumberOfDecreasesToday": 0,
      "WriteCapacityUnits": 1,
      "ReadCapacityUnits": 1
    },
    "TableSizeBytes": 0,
    "TableName": "ShoppingCart",
    "TableStatus": "CREATING",
    "KeySchema": [
      {
        "KeyType": "HASH",
        "AttributeName": "ItemName"
      }
    ],
    "ItemCount": 0,
    "CreationDateTime": 1475032237.808
  }
}
```

We could get that same output using the `describe-table` command at any time (except that the `TableStatus` parameter will change to `ACTIVE` very quickly after creation):

```
$ aws dynamodb describe-table \
  --table-name ShoppingCart
```

So now we've reserved a little nook and cranny in an AWS datacenter to hold our shopping cart data. But that `create-table` command is probably still a bit cryptic because there are some core concepts we haven't gone over yet. Let's start with supported data types.

DynamoDB's Data Types

DynamoDB's type system is, in essence, a stripped-down version of the type system that you'd find in a relational database. DynamoDB offers simple types such as Booleans, strings, and binary strings, but none of the more purpose-specific types that you'd find in, say, Postgres (such as currency

values or geometric types). DynamoDB offers these five *scalar types*, which can be thought of as its atomic types:

Type	Symbol	Description	JSON Example
String	S	A typical string like you'd find in most programming languages.	"S": "this is a string"
Number	N	Any integer or float. Sent as a string for the sake of compatibility between client libraries.	"N": "98", "N": "3.141592"
Binary	B	Base64-encoded binary data of any length (within item size limits).	"B": "4SrNYKrcv4wjczEf6u+TgaT2YaWGgU76YPhF"
Boolean	BOOL	true or false.	"BOOL": false
Null	NULL	A null value. Useful for missing values.	"NULL": true

Warning! No Empty Strings

String values cannot be empty in DynamoDB. You must provide some kind of placeholder when using strings, perhaps something like "nil" or "empty". This is frankly one of the oddest things about DynamoDB and something to always bear in mind when building applications using it.

In addition to the scalar types, DynamoDB also supports a handful of *set types* and *document types* (list and map):

Type	Symbol	Description	JSON Example
String set	SS	A set of strings.	"SS": ["Larry", "Moe"]
Number set	NS	A set of numbers.	"NS": ["42", "137"]
Binary set	BS	A set of binary strings.	"BS": ["TGFycnkK", "TW9lCg=="]
List	L	A list that can consist of data of any scalar type, like a JSON array. You can mix scalar types as well.	"L": [{"S": "totchos"}, {"N": "741"}]
Map	M	A key-value structure with strings as keys and values of any type, including sets, lists, and maps.	"M": {"FavoriteBook": {"S": "Old Yeller"}}

Set types act just like sets in most programming languages. All items in a set must be unique, which means that attempting to add the string "totchos" to a string set that already included it would result in no change to the set.

JSON in DynamoDB

And last but certainly not least, DynamoDB also allows you to store *any valid JSON* as an item attribute. So if your application's data model uses data types that are JSON serializable/deserializable, then you can store those types directly rather than wrangling them into the supported types you just saw. Keep two things in mind, though: Individual items can't be larger than 400 KB, so don't go too crazy using DynamoDB as a store for large JSON objects or BLOBs; and you can't query inside of JSON objects (your application will need to supply that logic).

DynamoDB's set of supported data types is fairly limited compared to what you would find in an RDBMS such as Postgres but quite rich in comparison with something like HBase, which only holds binary data and makes the application responsible for all serialization and deserialization.

Here's an example put operation that uses JSON:

```
$ aws dynamodb put-item --table-name Books \
  --item '{
    "Title": {"S": "Moby Dick"},
    "PublishYear": {"N": "2012"},
    "ISBN": {"N": "98765"},
    "PublisherInfo": {
      "Name": "Something"
    }
  }'
```

Basic Read/Write Operations

Now that you have a better sense of what actually goes on in tables and inside of items, you can begin actually working with data. Let's add a few items to our shopping cart using the put-item command:

```
$ aws dynamodb put-item --table-name ShoppingCart \
  --item '{"ItemName": {"S": "Tickle Me Elmo"}}'
$ aws dynamodb put-item --table-name ShoppingCart \
  --item '{"ItemName": {"S": "1975 Buick LeSabre"}}'
$ aws dynamodb put-item --table-name ShoppingCart \
  --item '{"ItemName": {"S": "Ken Burns: the Complete Box Set"}}'
```

As you can see, when we need to add data using the command line we need to send it across the wire as JSON. We now have three items in our shopping

cart. We can see a full listing using the scan command (which is the equivalent of an SQL `SELECT * FROM ShoppingCart` statement):

```
$ aws dynamodb scan \
  --table-name ShoppingCart
{
  "Count": 3,
  "Items": [
    {
      "ItemName": {
        "S": "1975 Buick LeSabre"
      }
    },
    {
      "ItemName": {
        "S": "Ken Burns: the Complete Box Set"
      }
    },
    {
      "ItemName": {
        "S": "Tickle Me Elmo"
      }
    }
  ],
  "ScannedCount": 3,
  "ConsumedCapacity": null
}
```

Scan operations involve all of the items in a table. It's perfectly fine to use them when you're not storing much data, but they tend to be very expensive options, so in a production table you should use them only if your use case absolutely requires processing every item in a table (and if you do require this, you may need to rethink your application logic!).

So how do you fetch, update, or delete specific items? This is where *keys* come in. In DynamoDB tables, you need to specify in advance which fields in the table are going to act as keys. In our case, our table has only one field, so the `ItemName` attribute will need to act as our key. But DynamoDB doesn't infer this automatically. This line in our create-table command specified the key: `--key-schema AttributeName=ItemName,KeyType=HASH`.

What happened here is that we told DynamoDB that we wanted the `ItemName` attribute to act as a key of type `HASH`. This means that we're using `ItemName` the way that we'd use keys in a standard key-value store like Redis: we simply provide DynamoDB with the "address" of the item and the database knows where to find it. In the next section, you'll see why keys in DynamoDB can also be much more complex—and powerful—than this.

For now, we can fetch specific items from our shopping cart, by key, using the `--key` flag:

```
$ aws dynamodb get-item --table-name ShoppingCart \
  --key '{"ItemName': {'S': 'Tickle Me Elmo'}}'
{
  "Item": {
    "ItemName": {
      "S": "Tickle Me Elmo"
    }
  }
}
```

As discussed in the intro to this chapter, DynamoDB enables you to specify whether or not you want to perform a consistent read on every request using the `--consistent-read` flag when you make a `get-item` request. This GET request would guarantee item-level consistency:

```
$ aws dynamodb get-item --table-name ShoppingCart \
  --key '{"ItemName': {'S': 'Tickle Me Elmo'}}' \
  --consistent-read
```

But let's be honest: Tickle Me Elmo isn't exactly all the rage these days so let's eliminate that from our cart (though we may regret it if Tickle Me Elmos experience a spike in resale value). We can do that on the basis of the hash key as well:

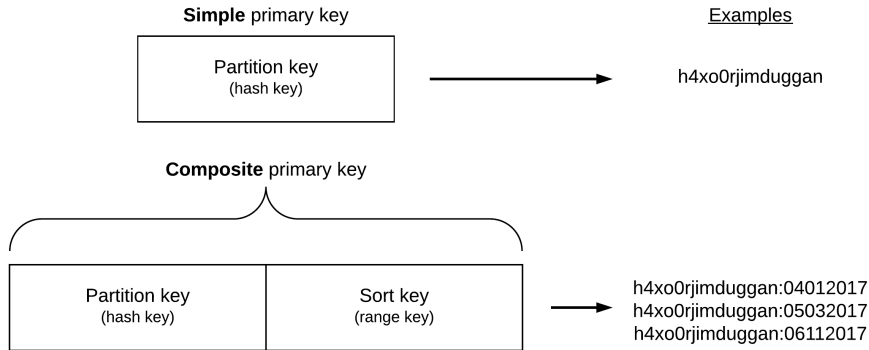
```
$ aws dynamodb delete-item --table-name ShoppingCart \
  --key '{"ItemName': {'S': 'Tickle Me Elmo'}}'
```

If we run the same scan operation as before, the `Count` field in the returned JSON will indicate that we now only have two items in our shopping cart.

Two Key Types, Many Possibilities

DynamoDB is, at root, a key-value store. But it's a special key-value store in that it provides two types of key that you can choose on a table-by-table basis. You can use either a *hash key* (aka partition key) by itself or you can use a *composite key* that combines a hash key and a *range key* (aka sort key), as shown in the [figure on page 224](#).

A hash key can be used to find items the same way you perform lookups in key-value databases such as HBase and Redis. You provide DynamoDB with a key and it checks the table to see if there's an item associated with the key. Imagine fetching information about NFL teams that have won the Super Bowl using the year as the hash key. Or imagine retrieving information about an



employee from a human resources database using the employee's Social Security Number as the hash key.

A combination partition and sort key, however, enables you to find items on the basis of the hash key if you know it in advance *or* to find multiple items via a *range query*.

Imagine a table storing information about books in which each book item has two properties: a title (string) and the year published (number). In this case, you could use the title as a hash key and the year published as a range key, which would enable you to fetch book data if you already know the title *or* if you only know a range of years that you're interested in.

Here's an example `aws` command that creates a table with that key combination:

```
$ aws dynamodb create-table \
  --table-name Books \
  --attribute-definitions AttributeName=Title,AttributeType=S \
    AttributeName=PublishYear,AttributeType=N \
  --key-schema AttributeName=Title,KeyType=HASH \
    AttributeName=PublishYear,KeyType=RANGE \
  --provisioned-throughput ReadCapacityUnits=1,WriteCapacityUnits=1
```

A book like *Moby Dick* has been published many times since its initial run. The structure of our Book table would enable us to store many items with a title of *Moby Dick* and then fetch specific items on the basis of which year the edition was published. Let's add some items to the table:

```
$ aws dynamodb put-item --table-name Books \
  --item '{
    "Title": {"S": "Moby Dick"},
    "PublishYear": {"N": "1851"},
    "ISBN": {"N": "12345"}
  }'
```

```
$ aws dynamodb put-item --table-name Books \
  --item '{
    "Title": {"S": "Moby Dick"},
    "PublishYear": {"N": "1971"},
    "ISBN": {"N": "23456"},
    "Note": {"S": "Out of print"}
  }'
$ aws dynamodb put-item --table-name Books \
  --item '{
    "Title": {"S": "Moby Dick"},
    "PublishYear": {"N": "2008"},
    "ISBN": {"N": "34567"}
  }'
```

You may have noticed that we supplied an ISBN attribute for each item without specifying that in the table definition. That's okay because remember that we only need to specify key attributes when creating tables. This gives DynamoDB its relative schemalessness, which you can also see at work in the second item we created, which has a Note attribute while the others do not.

To see which books were published after the year 1980, for example, we could use a range query:

```
$ aws dynamodb query --table-name Books \
  --expression-attribute-values '{
    ":title": {"S": "Moby Dick"},
    ":year": {"N": "1980"}
  }' \
  --key-condition-expression 'Title = :title AND PublishYear > :year'
{
  "Count": 1,
  "Items": [
    {
      "PublishYear": {
        "N": "2008"
      },
      "ISBN": {
        "N": "34567"
      },
      "Title": {
        "S": "Moby Dick"
      }
    }
  ],
  "ScannedCount": 1,
  "ConsumedCapacity": null
}
```

With the query command, the `--expression-attribute-values` flag enables us to provide values for variables that we want to use as JSON. The `--key-condition-expression`

flag enables us to provide an actual query string strongly reminiscent of SQL, in this case `Title = :title AND PublishYear > :year` (which becomes `Title = "Moby Dick" AND PublishYear > 1900` via interpolation).

If you've used a SQL driver for a specific programming language, then you've probably constructed parameterized queries like this. As output, we got one of three books currently in the Books table, as expected.

Whenever you provide key condition expressions, you must match directly on the hash key (in this case `Title = :title`) and then you can optionally provide a query for the range key using one of these operators: `=`, `>`, `<`, `>=`, `<=`, `BETWEEN`, or `begins_with`. A `BETWEEN a AND b` expression is the direct equivalent of `>= a AND <= b` and `begins_with` enables you to create an expression like `begins_with(Title, :str)` where `:str` could be a string.

But what if we wanted our result set to include only, say, ISBN data? After all, we know that all the books are titled Moby Dick, so we may not need that as part of our result set. We can tailor that result set using *attribute projection*. On the command line, you can perform attribute projection using the `--projection-expression` flag, which enables you to specify a list of attributes that you want returned for each item. This query would return only the ISBN for each edition of *Moby Dick* published after 1900:

```
$ aws dynamodb query --table-name Books \
  --expression-attribute-values \
    '{":title": {"S": "Moby Dick"}, ":year": {"N": "1900"}}' \
  --key-condition-expression \
    'Title = :title AND PublishYear > :year' \
  --projection-expression 'ISBN'
{
  "Count": 2,
  "Items": [
    {
      "ISBN": {
        "N": "23456"
      }
    },
    {
      "ISBN": {
        "N": "34567"
      }
    }
  ],
  "ScannedCount": 2,
  "ConsumedCapacity": null
}
```


If the attribute isn't defined for a specific item, then an empty object will be returned. Remember that only one of our book items has a Note attribute. Here's the result set when projecting for that attribute:

```
$ aws dynamodb query --table-name Books \
  --expression-attribute-values \
    '{":title": {"S": "Moby Dick"},":year": {"N": "1900"}}' \
  --key-condition-expression \
    'Title = :title AND PublishYear > :year' \
  --projection-expression 'Note'
{
  "Count": 2,
  "Items": [
    {
      "Note": {
        "S": "Out of print"
      }
    },
    {}
  ],
  "ScannedCount": 2,
  "ConsumedCapacity": null
}
```

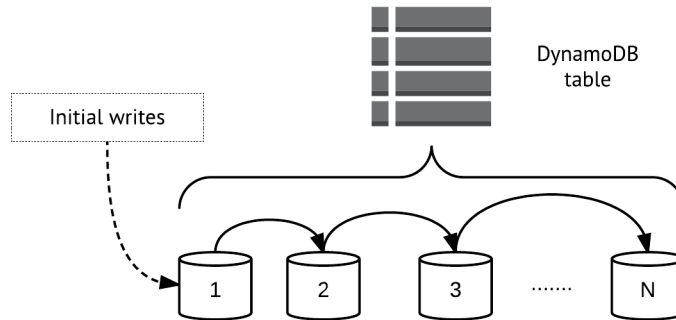
Note the empty object, {}, returned for one of the items.

As you can see in this section, choosing the right key setup for any DynamoDB table is extremely important because that will determine how your application is able to discover the items it's looking for. A good rule of thumb is this: If your application is built to know *in advance* where an item lives, then use just a hash key. An example here would be a user info database where you find items based on usernames already known to the application.

Spreading the Data Around: Partitioning in DynamoDB

Behind the scenes, DynamoDB's system for distributing item data across servers—*partitions* in DynamoDB parlance—is likely very complex. But from your perspective as a user it's really fairly simple. You can see a basic visual illustration of data partitioning in the [figure on page 228](#).

When you start writing data to a DynamoDB table, it begins filling in a first partition (partition 1 in the diagram). Eventually, that partition will fill up and data will start being distributed to partition 2, then at some point to partition 3, and so on out to N partitions. That N can be pretty much as large as you want; AWS provides no guidelines for limits to the number of partitions. The logic will simply repeat itself until you stop feeding data to the table.



How DynamoDB makes decisions about when to redistribute data between partitions isn't something that Amazon makes transparent to the user. It *just works*. The only thing you have to worry about is ensuring as even a distribution of partition keys as possible, as laid out in the next section.

So just how *big* are these partitions? That's determined by how much provisioned throughput you specify for the table. Think back to the read- and write-capacity units you specified in the create-table operation for the ShoppingCart table. There, one read capacity unit and one write capacity unit were specified. That provided the initial space allocation for the table. DynamoDB will add new partitions either when that first partition fills up or when you change your provisioned throughput settings in a way that's incompatible with the current settings.

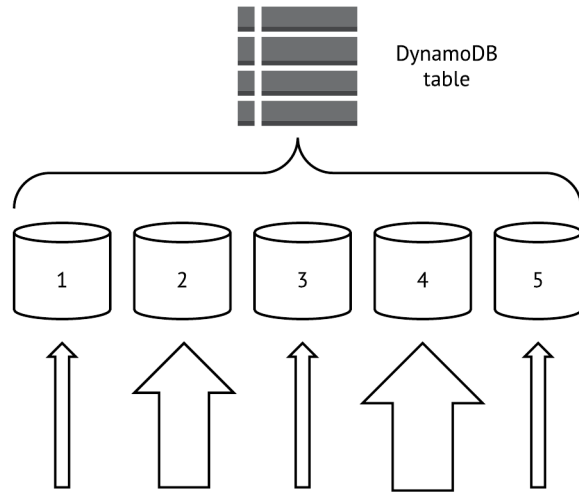
Location, Location, Location

As in any key-value store with limited querying capabilities, *where* you store things is an essential concern in DynamoDB and one that you should plan for from the get-go because it will have a huge impact on performance—and always remember that speedy, reliable performance is essentially the *whole point* of using DynamoDB.

As laid out in the previous section, DynamoDB uses a partition-based scheme for managing data within tables. You create a table and begin writing to it; over time, data is spread around across N partitions in an optimized way. But even though partitioning is automated, there are still some guidelines that you should follow when using tables.

The tricky part of partitioning is it's possible to create an uneven distribution of items across partitions. For a basic illustration, see the [figure on page 229](#).

DynamoDB *always* performs best—in terms of read and write speed—when access to partition keys is balanced. In this illustration, the size of each



upward-facing arrow shows how intensively each partition is accessed. Partitions 2 and 4 here take significantly more traffic than partitions 1, 3, and 5, making partitions 2 and 4 so-called “hotspots.” The presence of hotspots is likely to detract from performance for *all* partitions and should be avoided whenever possible.

In general, you can crystallize best practices for DynamoDB performance into just a few basic maxims.

First, if you’re using a hash key as your partition key, you should always strive for a data model in which the application *knows the key in advance* because this will enable you to target the item directly rather than relying on range and other queries. You should make the hash key something meaningful—for example, the username of each user in a CRM table, the device ID for each sensor in an “Internet of Things” table (more on that in Day 2), or a UUID-based transaction identifier in an inventory table. When working with key-value stores, this is simply the nature of the beast, and DynamoDB is no different here.

Second, because the partition key determines where data is stored, you should use partition keys that don’t cluster around just a few values. If you had a partition key with only a few possible values, for example good, bad, and ugly, you would produce hotspots around those values.

Third, in cases where you need to use a composite key—a hash key plus a range key—you should opt for *fewer partition keys and more range keys*. This is best illustrated with an example. Let’s say that you’re a video game company

that makes the game Bonestorm. Bonestorm has over a million players and you want to store daily total stats for each player on each day that they play the game. You could use username/date composite keys that look like this:

```
h4x0rjimduggan:04012017
h4x0rjimduggan:05032017
pitythefool:11092016
pitythefool:07082016
```

A less-optimal strategy here would be to make the date the hash key and usernames the sort key. Because there are far more users than there are dates—by a factor of thousands—the number of items with the same hash key would be much higher.

As is always the case with key-value stores, even a “key-value plus” store like DynamoDB, your application needs to be smart about how it stores data and how keys are structured. One of the great things about RDBMSs that we’ve discovered after years of experimenting with NoSQL databases is just how flexible they can be. DynamoDB is more flexible than other key-value stores—composite keys provide for that—but careful planning, before you write even your first items, is nonetheless essential.

Tables Within Tables: Indexes in DynamoDB

The last core component of DynamoDB that we’ll explore on Day 1 is *indexing*. Indexes are essentially mini tables that enable you to look up data more efficiently. Indexes in DynamoDB work much like they do in relational systems, storing lists of keys for objects based on some set of criteria.

Indexes usually offer a crucial performance boost because they prevent you from needing to run some of your more expensive queries. Instead of scanning an entire table and then selecting a subset of the results, which is almost never a good idea, you can *query or scan the index instead*. The price you pay for indexes is that they take up additional disk space. Whether or not that price is worth the performance boost depends on your use case and is difficult to decide in advance without knowing a lot of particulars. In DynamoDB, indexes come in two flavors: local and global.

Local Secondary Indexes

Local secondary indexes (LSIs) let you query or scan attributes *outside of your primary hash and sort key*. The “local” in “local secondary index” means *among items sharing a partition key*. You can see an example of this in the following figure. Here, a local secondary index is created for the High Score attribute.

The LSI on the High Score attribute here means that we can perform scans or queries related to high scores *for a specific user*.

Username (hash key)	Date (range key)	High score	Avatar
h4x0rjimduggan	04012017	751	Lion
h4x0rjimduggan	05032017	852	Skull
beboprocksteady75	04262017	451	Raptor
beboprocksteady75	05032017	562	Lion
beboprocksteady75	06122017	567	Narwhal

Here's an example range query for the HighScore attribute that shows all of the dates on which the player h4x0rjimduggan scored over 800 points:

```
$ aws dynamodb query --table-name BonestormData \
  --expression-attribute-values \
    '{":user": {"S": "h4x0rjimduggan"}, ":score": {"N": "800"}}' \
  --key-condition-expression \
    'Username = :user AND HighScore > :score' \
  --projection-expression 'Date'
```

Always bear in mind that *LSIs can't be modified after a table has been created*, so be extra careful to plan for them in advance (or, even better, thoroughly test different indexing models in development).

Global Secondary Indexes

Global secondary indexes (GSIs) are indexes that aren't restricted to items with the same partition key. The “global” here essentially means “any attribute in the table.” See the [figure on page 232](#) for an example. You can find *any* items in the table using the Avatar attribute.

Here's an example query that finds all items for which the Avatar field equals Lion and returns the Username and Date for each of those items:

```
$ aws dynamodb query --table-name BonestormData \
  --expression-attribute-values \
    '{":avatar": {"S": "Lion"}}' \
  --key-condition-expression \
    'Avatar = :avatar' \
  --projection-expression 'Username, Date'
```

Username (hash key)	Date (range key)	High score	Avatar
h4x0rjimduggan	04012017	751	Lion
h4x0rjimduggan	05032017	852	Skull
beboprocksteady75	04262017	451	Raptor
beboprocksteady75	05032017	562	Lion
beboprocksteady75	06122017	567	Narwhal

An important thing to note is that GSIs can be modified *after* a table has been created, whereas LSIs cannot. This makes it somewhat less crucial to get your GSIs right from the get-go, but it's still never a bad idea to make a thorough indexing plan in advance.

Day 1 Wrap-Up

Here on Day 1 we did a *lot* of conceptual exploration, from supported data types to indexing to the core philosophy of DynamoDB, but we didn't really *do* anything all that exciting with DynamoDB. That will change on Day 2, when we'll embark on one of the most ambitious practical projects in this book, building a streaming data pipeline that ties together multiple AWS services to create a streaming data pipeline that uses DynamoDB as a persistent data store.

Day 1 Homework

Find

1. DynamoDB does have a specific formula that's used to calculate the number of partitions for a table. Do some Googling and find that formula.
2. Browse the documentation for the DynamoDB streams feature.³
3. We mentioned limits for things like item size (400 KB per item). Read the Limits in DynamoDB documentation⁴ to see which other limitations apply when using DynamoDB so that you don't unknowingly overstep any bounds.

3. <http://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html>

4. <http://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Limits.html>

Do

1. Using the formula you found for calculating the number of partitions used for a table, calculate how many partitions would be used for a table holding 100 GB of data and assigned 2000 RCUs and 3000 WCUs.
2. If you were storing tweets in DynamoDB, how would you do so using DynamoDB's supported data types?
3. In addition to PutItem operations, DynamoDB also offers *update item* operations that enable you to modify an item if a *conditional expression* is satisfied. Take a look at the documentation for conditional expressions⁵ and perform a conditional update operation on an item in the ShoppingCart table.

Day 2: Building a Streaming Data Pipeline

On Day 1, we made some solid progress on understanding the core concepts of DynamoDB, from how DynamoDB partitions data to the data types that it supports to secondary indexes. We worked through some simple table management commands and some basic CRUD operations on that table to see the concepts in action.

We were exposed to some features that set DynamoDB apart from other, more strict key-value stores, but those features are mostly garnishes. The main dish is DynamoDB's unique blend of extreme scalability, predictably solid performance as you scale out, and freedom from operational burdens.

On Day 2, we'll build something that can take full advantage of those features in a way that we couldn't in our Day 1 CRUD exercises. We'll build a streaming data pipeline that pumps sensor data into DynamoDB (we'll use a Ruby script to act as mock sensors). Those mock sensors will populate our table with three pieces of data per "reading":

- An identifier for the sensor—for example, sensor-1 or temp-a1b2c3
- A timestamp for the reading in Unix time (this will allow for easier sorting)
- The current temperature picked up by the sensor—for example, 81.4 or 73.2

But instead of writing to DynamoDB directly, we'll use a streaming data service called Kinesis that's built for handling massive write throughput; messages passing through Kinesis will be written to DynamoDB using functions deployed to AWS's Lambda service.

It may seem a bit odd to focus on connecting AWS services in a chapter devoted to DynamoDB. But it's important to note that cloud databases are

5. <http://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Expressions.ConditionExpressions.html>

rarely used as standalone tools. Cloud providers typically offer lots and lots of services that can run in the same datacenter as your database, built to supplement one another. To use multiple services together is to go with the grain of cloud computing.

A Data Model for Sensor Data

The data pipeline that we build later today will be capable of handling *lots* of streaming data, to the tune of many megabytes per second. But before we open up that pipeline, we need to make some big choices about our data model. For reasons we went over on Day 1, getting data modeling right is *everything* in DynamoDB and any missteps can get you into trouble, especially in a big production system.

Getting Our Keys Right

Keys are extremely important in DynamoDB because they not only act as “addresses” for items, as in systems like Redis, in DynamoDB they also determine *how* your data is distributed between partitions, which in turn affects the write/read performance of the table.

We’ll call our DynamoDB table `SensorData`, and it will consist of the following columns:

- The hash key will be a `ReadingId` attribute that will be randomly generated. This will ensure that our partition keys are evenly distributed across partitions. There will be no range key, as each item will have a different partition key; range keys can only sort across items with the same partition key, and in this case a range key for a single partition key would be nonsensical.
- The ID of the device emitting the data (`sensor1`, `factory-sensor-1337`, and so on).
- A timestamp number expressing the current Unix time.
- The temperature for the reading.

When creating DynamoDB tables via the command line, you can either use flags (as for the previous `ShoppingCart` table) or you can use a JSON specification. The spec for our `SensorData` table will be complex enough that using CLI flags will be too unwieldy. So we’ll do it this way instead:

```
$ aws dynamodb create-table \
  --cli-input-json file://sensor-data-table.json
```

And here’s the JSON spec itself:

dynamodb/sensor-data-table.json

```

{
  "TableName": "SensorData",
  "KeySchema": [
    {
      "AttributeName": "SensorId",
      "KeyType": "HASH"
    },
    {
      "AttributeName": "CurrentTime",
      "KeyType": "RANGE"
    }
  ],
  "AttributeDefinitions": [
    {
      "AttributeName": "SensorId",
      "AttributeType": "S"
    },
    {
      "AttributeName": "CurrentTime",
      "AttributeType": "N"
    },
    {
      "AttributeName": "Temperature",
      "AttributeType": "N"
    }
  ],
  "LocalSecondaryIndexes": [
    {
      "IndexName": "TemperatureIndex",
      "KeySchema": [
        {
          "AttributeName": "SensorId",
          "KeyType": "HASH"
        },
        {
          "AttributeName": "Temperature",
          "KeyType": "RANGE"
        }
      ]
    }
  ],
  "Projection": {
    "ProjectionType": "ALL"
  },
  "ProvisionedThroughput": {
    "ReadCapacityUnits": 2,
    "WriteCapacityUnits": 2
  }
}

```

Keep in mind that this data model is highly specific to our use case. Many use cases may involve more complex column structures, more secondary indexes, more attributes, and so on. The important thing is making sure that you go through a similar decision-making process whenever you use DynamoDB; missteps here will cost you, and we do mean actual money.

Navigating Trade-offs in Performance Tuning

As we’ve stressed before, you simply shouldn’t use DynamoDB if you’re after anything but extremely good performance at massive scale. But as with any complex system, performance and scalability in DynamoDB are *not* guaranteed. There’s a wide variety of missteps that will ensure that you have a deeply sub-optimal experience, spend too much money, or both.

On Day 1, we talked about performance optimization from the standpoint of data modeling and how the distribution of primary keys is very important. In this section, the focus will be more operational. DynamoDB provides a small handful of knobs you can adjust that can have a big impact on performance. These knobs are the closest thing you get to “ops” in DynamoDB, but they aren’t terribly complex. You just need to keep a few basic principles in view.

Throughput is the central concept here. When we created the ShoppingCart table on Day 1, we used this command but didn’t provide much explanation of the `--provisioned-throughput` flag:

```
$ aws dynamodb create-table --table-name ShoppingCart \
  --attribute-definitions AttributeName=ItemName,AttributeType=S \
  --key-schema AttributeName=ItemName,KeyType=HASH \
  --provisioned-throughput ReadCapacityUnits=1,WriteCapacityUnits=1
```

Read capacity and write capacity units are handled differently in DynamoDB. Write capacity units (WCUs) are measured in units of 1 KB per second while read capacity units (RCUs) are measured in units of 4 KB per second. You can set both read and write throughput on a per-table basis. For workloads that are more write heavy, you could provision just one RCU and many WCUs, or vice versa for read-heavy workloads.

We won’t delve too far into DynamoDB throughput tuning here. Just keep a few things in mind:

- Strive to match the RCUs and WCUs in a table to balance between read and write intensity. Perform regular checks—Amazon offers numerous services to help with this, such as CloudWatch—to ensure that you haven’t over- or under-provisioned throughput on either the read or the write side. Perhaps even set up an automated system to re-provision when necessary.

- Considerations about read and write throughput *include secondary indexes*, both local and global. Throughput should be provisioned based on the size of index entries, *not* the size of the actual table items.
- Strongly consistent reads are about twice as expensive as eventually consistent reads.

If you're ever concerned about costs, the DynamoDB console on the AWS website provides an interface that you can use to get cost estimates, as shown in the figure that follows.

The screenshot shows the AWS DynamoDB console interface for a table named 'SensorData'. The 'Capacity' tab is selected, showing the 'Provisioned capacity' section. It displays 'Read capacity units' and 'Write capacity units' both set to 1. The 'Estimated cost' is \$0.59 / month. There are 'Save' and 'Cancel' buttons at the bottom.

Throughput settings for a table are *not* fixed for all time and can be updated at will. For our purposes here, 1 unit of RCU and WCU will suffice for our table. But we couldn't let you proceed beyond this section without mentioning this second, essential element of DynamoDB performance.

Now that we have a SensorData table ready to accept writes, we can get back to building our streaming sensor data pipeline.

DynamoDB Auto Scaling

An alternative to manually setting parameters like RCUs and WCUs is to use DynamoDB's Auto Scaling feature.

```
$ aws application-autoscaling register-scalable-target \
  --service-namespace dynamodb \
  --resource-id "table/ShoppingCart" \
  --scalable-dimension "dynamodb:table:WriteCapacityUnits" \
  --min-capacity 5 \
  --max-capacity 10 \
  --role-arn arn:aws:iam::123456654321:policy/MyAutoscalingPolicy
```

It's a Streaming World

Data *streams* are best understood in contrast to data *batches*. When you process a batch—maybe a huge CSV file or a massive Postgres table—you

take a well-defined mass of data and apply some kind of logic to it, for example selecting all values that satisfy a WHERE clause in an SQL query or parsing all the values in a column in an Excel spreadsheet.

When you process a stream of data, however, you don't know in advance how many items will pass through the stream for processing. Instead, you design some kind of processing logic, open up a pipe, and handle things as they come through. Although there's still plenty of room for batch processing in today's data landscape, streaming models have become more popular with the predominance of realtime inputs from devices such as smartphones and, in our case, sensors.

A Realtime, Streaming, Functional Data Pipeline

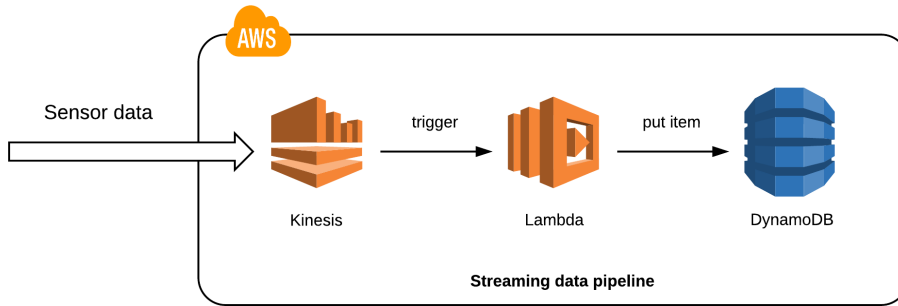
For today's practical exercise, we'll use a combination of three AWS services—DynamoDB, Lambda, and Kinesis—to build a data pipeline that's capable of accepting *lots* of incoming data, processing it, and finally storing it in DynamoDB.

Amazon's *Kinesis* is a service that enables you to manage data streams that you can pipe to any number of destinations, from S3 buckets to Redshift warehouses to, in our case, AWS Lambda functions (more on that in a minute). You can think of Kinesis as a managed, cloud-only version of Apache Kafka, and if you've worked with Kafka then some of the core primitives driving Kinesis may be familiar to you already: topics, producers, consumers, records, and so on. Not a perfect analogy, but it should suffice for our exercise.

Lambda is a Functions-as-a-Service (FaaS) platform that enables you to manage functions that run in Amazon datacenters without *any* concern for running servers or managing infrastructure. The popularity of Lambda has spawned the so-called “serverless” paradigm of computing, which is a bit of misnomer in that Lambda functions do, of course, run on servers, but the fact that you simply write functions that process inputs however you'd like and upload them to AWS makes it at least *feel* serverless.

What we build today will ultimately look like the [figure on page 239](#).

Data will flow through a Kinesis stream to a Lambda function that then processes the data. The Lambda processing step writes the incoming data to a DynamoDB table. Once we've built this, we'll feed some example data into Kinesis using the aws CLI tool to test things and then start pumping sensor data in.



Before you start going through the CLI examples here, do two things. First, `cd` into the `code/dynamodb` folder for the DynamoDB chapter. Then, create a Kinesis stream using the `aws` tool:

```
$ export STREAM_NAME=temperature-sensor-data
$ aws kinesis create-stream \
  --stream-name ${STREAM_NAME} \
  --shard-count 1
```

When creating this stream, we gave it a name and specified a *shard count*. The shard count essentially determines how much parallel processing power you want your stream to have. More shards means more power but also more cost, so we'll use just one shard for our example here as it's all we'll really need. Now let's have a look at how AWS sees our stream:

```
$ aws kinesis describe-stream \
  --stream-name ${STREAM_NAME}
{
  "StreamDescription": {
    "RetentionPeriodHours": 24,
    "StreamName": "iot-temperature-data",
    "Shards": [],
    "StreamARN": "arn:aws:kinesis:...:stream/iot-temperature-data",
    "EnhancedMonitoring": [
      {
        "ShardLevelMetrics": []
      }
    ],
    "StreamStatus": "CREATING"
  }
}
```

At first, the status of the stream will be `CREATING`, but within a few seconds the status should change to `ACTIVE`. Once that happens, you can write some

data to the stream. Each record that you write to Kinesis needs to contain two things: a partition key and a binary large object (BLOB) of data. The partition key determines which shard the record is written to (as with DynamoDB's partitioning system). Set the `STREAM_ARN` environment variable to the StreamARN shown in the output, which should begin with `arn:aws:kinesis`.

Here's an example write to the `temperature-sensor-data` stream that we just created:

```
$ aws kinesis put-record \
  --stream-name ${STREAM_NAME} \
  --partition-key sensor-data \
  --data "Baby's first Kinesis record"
{
  "ShardId": "shardId-000000000000",
  "SequenceNumber": "4957192301...3577493926"
}
```

Don't worry too much about partitions, shards, and the returned JSON here, as we won't delve too deeply into Kinesis in this book. The important thing to know for now is that our stream is ready to accept incoming records and pass them along to the Lambda function that we'll create later.

Now one essential piece of our pipeline is in place, but there are more puzzle pieces we need to connect. Kinesis is a powerful system but it's really just a fancy pipe. It doesn't actually *process* or write data, so we can't use it to actually store items in DynamoDB. We'll need to create a processing layer that ingests records from Kinesis and actually does something with them.

Up until very recently, the default way to create this kind of processing layer in AWS would be to write an application that can handle incoming streams of data and run that application on Elastic Compute Cloud (EC2) servers. That's a perfectly reasonable approach, but it requires you to fire up and manage servers. We can avoid that by using Lambda, which requires us to simply write some code, run some basic CLI commands, and let AWS do the work.

First, we need to write our function. Lambda supports several languages for Lambda functions, but let's use JavaScript because its callback-based logic is a natural fit for Lambda (and perhaps even an inspiration for Lambda!). Here is the code for our handler:

```
dynamodb/ProcessKinesisRecords.js
var AWS = require('aws-sdk');
var DynamoDB = new AWS.DynamoDB({
  apiVersion: '2012-08-10',
  region: 'us-east-1',
});
```

```

exports.kinesisHandler = function(event, context, callback) {
  // We only need to handle one record at a time
  var kinesisRecord = event.Records[0];

  // The data payload is base 64 encoded and needs to be decoded to a string
  var data =
    Buffer.from(kinesisRecord.kinesis.data, 'base64').toString('ascii');
  // Create a JSON object out of that string
  var obj = JSON.parse(data);
  var sensorId = obj.sensor_id,
      currentTime = obj.current_time,
      temperature = obj.temperature;

  // Define the item to write to DynamoDB
  var item = {
    TableName: "SensorData",
    Item: {
      SensorId: {
        S: sensorId
      },
      CurrentTime: {
        // Remember that all numbers need to be input as strings
        N: currentTime.toString()
      },
      Temperature: {
        N: temperature.toString()
      }
    }
  };

  // Perform a put operation, logging both successes and failures
  DynamoDB.putItem(item, function(err, data) {
    if (err) {
      console.log(err, err.stack);
      callback(err.stack);
    } else {
      console.log(data);
      callback(null, data);
    }
  });
}

```

When writing any Lambda handler function, you are given three JavaScript objects to work with when your function is triggered:

- The event object contains the data that has been passed to the function, in our case the JSON object written to Kinesis.
- The context object holds information about the environment in which the function is running.

- The callback object signals that the operation is finished. If called with a single argument, that means that the function returns an error; if called with null and a string, then the function is deemed successful and the string represents the success message.

The event object that our function receives from Kinesis is a JSON object with a `Records` field that will hold the data for each record. In our pipeline, Kinesis will only deliver a single record at a time. For each record that arrives, we'll put an item to our `SensorData` table.

Before we can stitch Kinesis and Lambda together, however, we need to create an AWS *security role* that enables us to do that. Role management in AWS is handled by a service called Identity and Access Management, or IAM. If you've managed users, roles, and groups in database systems such as Postgres, the concepts here are quite similar. The following set of commands will:

- Create the role using a JSON document that you can peruse in the `lambda-kinesis-role.json` file.
- Attach a *policy* to that role that will enable Kinesis to pass data to Lambda.
- Store the ARN for this role in an environment variable.

```
$ export IAM_ROLE_NAME=kinesis-lambda-dynamodb
$ aws iam create-role \
  --role-name ${IAM_ROLE_NAME} \
  --assume-role-policy-document file://lambda-kinesis-role.json
$ aws iam attach-role-policy \
  --role-name ${IAM_ROLE_NAME} \
  --policy-arn \
    arn:aws:iam::aws:policy/service-role/AWSLambdaKinesisExecutionRole
$ aws iam attach-role-policy \
  --role-name ${IAM_ROLE_NAME} \
  --policy-arn \
    arn:aws:iam::aws:policy/AmazonDynamoDBFullAccess
```

Now you need to get the ARN for that role. Run this command and set the `ROLE_ARN` environment variable to the proper `Arn`, which should begin with `arn:aws:iam:`:

```
$ aws iam get-role --role-name ${IAM_ROLE_NAME}
```

In order to upload our function to Lambda, we need to create a Zip file out of it and then upload the zipped payload using the `create-function` command:

```
$ zip ProcessKinesisRecords.zip ProcessKinesisRecords.js
$ aws lambda create-function \
  --region us-east-1 \
  --function-name ProcessKinesisRecords \
```



```

--zip-file fileb://ProcessKinesisRecords.zip \
--role ${ROLE_ARN} \
--handler ProcessKinesisRecords.kinesisHandler \
--runtime nodejs6.10
{
  "CodeSha256": "LdmN2sMF5kwdZiRbIYAtdhs4J8pX39Qa6EhvGdGAcOQ=",
  "FunctionName": "kinesis-dynamodb-processor",
  "CodeSize": 530,
  "MemorySize": 128,
  "FunctionArn": "arn:aws:lambda:...",
  "Version": "$LATEST",
  "Role": "arn:aws:iam:...",
  "Timeout": 3,
  "LastModified": "2017-04-13T04:07:36.833+0000",
  "Handler": "kinesisHandler",
  "Runtime": "nodejs4.3",
  "Description": ""
}

```

If you have any modifications that you want to make to the function on your own, you can update it at any time by creating a new Zip file and using the `update-function-code` command:

```

$ zip ProcessKinesisRecords.zip ProcessKinesisRecords.js
$ aws lambda update-function-code \
  --function-name ProcessKinesisRecords \
  --zip-file fileb://ProcessKinesisRecords.zip

```

We can run a test invocation of the new function using a mock input from a text file and storing the output in another text file (that you can peruse on your own):

```

$ aws lambda invoke \
  --invocation-type RequestResponse \
  --function-name ProcessKinesisRecords \
  --payload file://test-lambda-input.txt \
  lambda-output.txt
{
  "StatusCode": 200
}

```

Success! We've triggered a Lambda function using a mock data input payload mimicking a Kinesis stream. Lambda then handled the data, turned it into a DynamoDB item, and made a successful write. What's missing now is that Kinesis isn't yet able to trigger and pass data to our Lambda function. For that, we need to create a *source mapping* that tells AWS that we want our `iot-temperature-data` to trigger our `ProcessKinesisRecords` Lambda function whenever a record passes into the stream. This command will create that mapping:

```
$ aws lambda create-event-source-mapping \
  --function-name ProcessKinesisRecords \
  --event-source-arn ${KINESIS_STREAM_ARN} \
  --starting-position LATEST
{
  "UUID": "0f56a8c7-de6d-4a77-b536-9ec87be5a065",
  "StateTransitionReason": "User action",
  "LastModified": 1492057092.585,
  "BatchSize": 100,
  "EventSourceArn": "arn:aws:kinesis:...",
  "FunctionArn": "arn:aws:lambda:...",
  "State": "Creating",
  "LastProcessingResult": "No records processed"
}
```

You can see a description of the event source mapping:

```
$ aws lambda list-event-source-mappings
{
  "EventSourceMappings": [
    {
      "UUID": "0f56a8c7-de6d-4a77-b536-9ec87be5a065",
      "StateTransitionReason": "User action",
      "LastModified": 1492057140.0,
      "BatchSize": 100,
      "State": "Enabled",
      "FunctionArn": "arn:aws:lambda:...",
      "EventSourceArn": "arn:aws:kinesis:...",
      "LastProcessingResult": "No records processed"
    }
  ]
}
```

If the value of State isn't yet Enabled, wait a few seconds and that should change.

At this point, the pipeline is in place. Records that are written to the temperature-sensor-stream stream in Kinesis will be processed by a Lambda function, which will write the results to the SensorData table in DynamoDB. Just to be sure, let's put a record to the stream and see what happens:

```
$ export DATA=$(
  echo '{
    "sensor_id":"sensor-1",
    "temperature":99.9,"current_time":123456789
  }' | base64)
$ aws kinesis put-record \
  --stream-name ${STREAM_NAME} \
  --partition-key sensor-data \
  --data ${DATA}
```

That should return a JSON object with a `ShardId` and `SequenceNumber` as before. But let's see if that mock temperature sensor reading ended up in DynamoDB:

```
$ aws dynamodb scan --table-name SensorData
{
  "Count": 1,
  "Items": [
    {
      "CurrentTime": {
        "N": "123456789"
      },
      "Temperature": {
        "N": "99.9"
      },
      "SensorId": {
        "S": "sensor-1"
      }
    }
  ]
}
```

Success again! Our pipeline is now all set up and ready to handle sensor data inputs. On Day 3, we'll begin pumping data into Kinesis and letting Lambda do its magic.

Day 2 Homework

Find

1. Read some documentation for the DynamoDB Streams feature (not to be confused with Kinesis).⁶ Think of a compelling use case for this feature.
2. Find one or more DynamoDB client libraries for your favorite programming language. Explore how to perform CRUD operations using that library.
3. DynamoDB supports object expiry using time to live (TTL). Find some documentation on TTL in DynamoDB and think of some use cases for it.

Do

1. One way to improve the performance of Kinesis is to write records to different partitions. Find some documentation on partitioning in Kinesis and think of a stream partitioning scheme that would work with our sensor data model (rather than writing all records to the sensor-data partition).

6. <http://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html>

2. Modify various elements of our data pipeline—the `SensorData` table definition, the Lambda function, and so on—to enable sensors to write humidity-related data to the pipeline (as a percentage). Make sure that *all* of these components line up properly!

Day 3: Building an “Internet of Things” System Around DynamoDB

On Day 2, we delved into DynamoDB’s streaming feature and turned our DynamoDB database from a fairly standard cloud storage mechanism into a much more dynamic processing system consisting of a Kinesis-Lambda-DynamoDB system. What we *didn’t* do on Day 2 was provide any data inputs beyond some CLI test runs. On Day 3, we’ll provide some “Internet of Things”-style data from mock sensors.

The term “Internet of Things” (aka IoT) is kind of like “Big Data” or “the Cloud.” Its use is controversial because there’s a lot of buzz surrounding it, and what it points to isn’t so much something that already exists as much as it is a set of emerging technological possibilities. The “things” in the expression can refer to everything from moisture sensors in your refrigerator to a heart-rate monitor you wear on your wrist—essentially anything that isn’t a “normal” computer (laptop, smartphone, cloud server) and is capable of producing data and transmitting it to some part of the Internet.

The interesting thing about the IoT from the perspective of this book is that things like sensors can produce *tons* of data, especially when aggregating across many data-producing devices. Imagine a 50-acre vineyard with moisture and temperature sensors hooked up every few feet along the rows of vines, or a high-tech manufacturing facility where each machine in an assembly line is feeding all kinds of data to some centralized source. In both cases, these devices might be sending data every few hours or they might be sending it more than once a second. IoT data usually comes in small chunks (maybe just a few sensor readings at a time), but those small chunks can really add up.

DynamoDB is a good choice for an IoT database because it’s built to handle not just huge data sets but data sets whose size can’t be determined in advanced. The downside of DynamoDB, as we discussed before, is that it isn’t built for complex queries of the type you’d be able to run on a relational database. For our needs here, though, DynamoDB’s “key-value plus” querying system will do just fine. Our tables will be fairly simple, and range queries will enable us to use DynamoDB as a powerful timeseries database. Any processing logic we need to apply beyond this can happen on the application side if we need it to.

Mock Data Inputs

In a real sensor data setup, you would likely use a protocol like Message Queue Telemetry Transport (MQTT)⁷ as a device messaging protocol to feed data into a streaming data pipeline like ours, perhaps in conjunction with a broker interface like Amazon’s IoT service.⁸

Here, we’ll do something a bit simpler that doesn’t require having actual sensor devices on hand. We’ll use a Ruby script that auto-generates temperature data and writes that data to our data pipeline as JSON.

```
dynamodb/upload-sensor-data.rb
require 'aws-sdk'
require 'random-walk'
require 'time'

STREAM_NAME = 'temperature-sensor-data'

# Make sure that both a sensor ID and number of iterations are entered
if ARGV.length != 2
  abort("Must specify a sensor ID as the first arg and N as the second")
end

@sensor_id = ARGV[0]
@iterator_limit = ARGV[1].to_i

# The Kinesis client object. Supply a different region if necessary
@kinesis_client = Aws::Kinesis::Client.new(region: 'us-east-1')

# An array used to generate random walk values
@temp_walk_array = RandomWalk.generate(6000..10000, @iterator_limit, 1)

# The iterator starts at 0
@iterator = 0

def write_temp_reading_to_kinesis
  # Generate a random current temperature from the walk array
  current_temp = @temp_walk_array[@iterator] / 100.0

  # The JSON payload for the reading
  data = {
    :sensor_id    => @sensor_id,
    :current_time => Time.now.to_i,
    :temperature => current_temp,
  }

  # The record to write to Kinesis
  kinesis_record = {
    :stream_name => STREAM_NAME,
    :data        => data.to_json,
  }
```

7. <http://mqtt.org>

8. <https://aws.amazon.com/iot>

```

    # We'll use just a single partition key here
    :partition_key => 'sensor-data',
  }

  # Write the record to Kinesis
  @kinesis_client.put_record(kinesis_record)

  puts "Sensor #{@sensor_id} sent a temperature reading of #{current_temp}"

  @iterator += 1

  # Exit if script has iterated N times
  if @iterator == @iterator_limit
    puts "The sensor has gone offline"
    exit(0)
  end
end

while true
  write_temp_reading_to_kinesis
  # Pause 2 seconds before supplying another reading
  sleep 2
end

```

In order to run the script, you'll need to install the `random-walk` and `aws-sdk` gems:

```
$ gem install random-walk aws-sdk
```

This script requires you to specify a sensor identifier, such as `sensor-1` or whatever you'd like, as well as the number of times that you'd like the script to write data to the pipeline. To make the `temp-sensor-1` sensor write 1,000 times to the pipeline:

```
$ ruby upload-sensor-data.rb temp-sensor-1 1000
```

A randomized temperature reading will be written to the pipeline every 2 seconds until it's gone through the supplied number of iterations. The temperature readings will be between 60 and 100 degrees and based on a *random walk* pattern, which means that the readings will follow a lightly meandering path rather than being truly random. A four-step random walk pattern would produce a series like 47.2, 47.3, 47.7, 47.6 as opposed to something like 47.2, 68.9, 50.3, 32.1. The script will also supply a Unix timestamp to each reading (which we can later use to run range queries).

Open up multiple shell sessions and run the script for multiple device IDs. Or use this command, which will write 1,000 sensor readings each for 10 sensors labeled `sensor-1`, `sensor-2`, and so on.

```
$ for n in {1..10}; do
  ruby upload-sensor-data.rb sensor-{$n} 1000 &
done

```

Let that run for as long as you’d like. You can shut down all the running mock sensors at any time:

```
$ pgrep -f upload-sensor-data | xargs kill -9
```

Scan the SensorData table a few times to see the number of items steadily increasing:

```
$ aws dynamodb scan --table-name SensorData
```

As the table fills up, scan operations will become increasingly slow. Now would be a good time to explore local secondary index queries. Remember that when we created the SensorData table we created a local secondary index for the Temperature attribute. Get two timestamp values for two temperature readings—the first and the 201st—like this:

```
$ T1=$(aws dynamodb scan --table-name SensorData \
  --query Items[0].CurrentTime.N | tr -d ' ')
$ T2=$(aws dynamodb scan --table-name SensorData \
  --query Items[200].CurrentTime.N | tr -d ' ')
```

Then run this query, substituting the values for T1 and T2 (make sure to use the larger of the two values for t2).

```
$ aws dynamodb query --table-name SensorData \
  --expression-attribute-values '{
    ":t1": {"N": "..."},
    ":t2": {"N": "..."},
    ":sensorId": {"S": "sensor-1"}}
  }' \
  --key-condition-expression \
    'SensorId = :sensorId AND CurrentTime BETWEEN :t1 AND :t2' \
  --projection-expression 'Temperature'
```

That should return all the Temperature values for the sensor-1 sensor between the two timestamps (feel free to substitute a different name if you used a different naming scheme).

Now see how many items are currently in the table:

```
$ aws dynamodb scan --table-name SensorData \
  --query Count
```

At this point, you may be up to several hundreds of items. Let the count get above 1,000 or so and then stop the sensors using the kill -9 command we used before. At that point, there will be plenty of data in the SensorData table to move on to the next exercise.

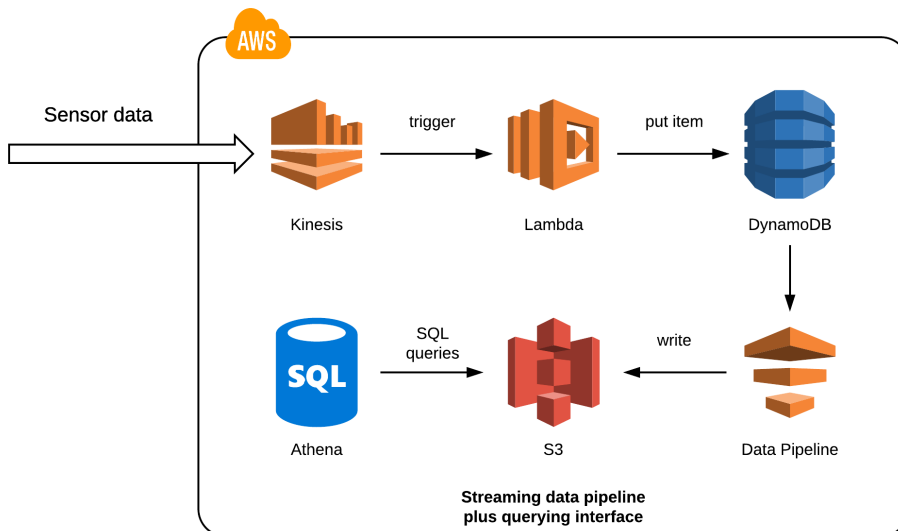
An SQL Querying Interface

Our mock sensors have now streamed over 1,000 entries into our DynamoDB table. We've seen one way of accessing that data, which is fetching sensor readings based on a time range. This is a cool capability, but if we wanted to gather more meaningful metrics for our sensor data, we'd need a more powerful interface. What if you had, say, data analysts or climate scientists on your team that needed to perform much more complex queries or mathematical calculations using that sensor data?

One way that you can do this on AWS is to use plain old SQL. Now, don't get too excited just yet, because you can't perform SQL queries over DynamoDB tables *directly*—at least not yet—but what you *can* do instead is transfer data stored in DynamoDB tables into an S3 (Simple Storage Service) bucket using a service called Data Pipeline and then run SQL queries directly against that data using another AWS service called Athena.

Time for Another Pipeline

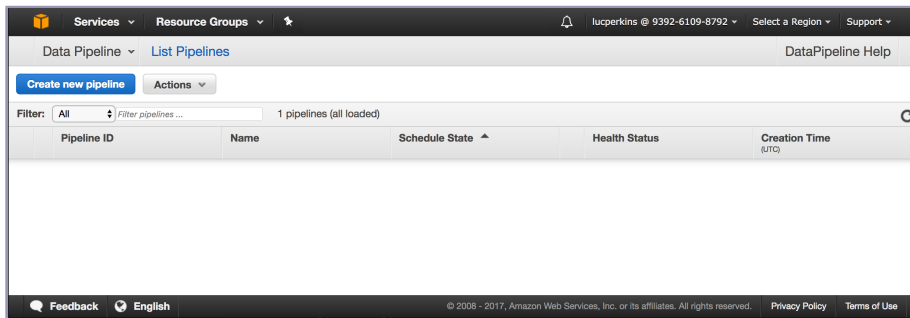
Amazon's Data Pipeline service enables you to create batch jobs that efficiently move data between Amazon services (including S3, the Redshift data warehousing service, and many more). You can write your own pipeline logic from scratch or you can use a template for ease of use. Fortunately for us, there's a predefined *Export DynamoDB table to S3* pipeline definition that will make this very simple. Ultimately, our streaming pipeline plus querying interface will look like the architecture in the figure that follows.



To begin, you’ll need to create a new S3 bucket to store JSON data from the `SensorData` table in DynamoDB. You can call this bucket whatever you’d like, so long as it’s globally unique (as in unique among *all* AWS users). Perhaps call it `sensor-data`, and add your local username and use the `aws` tool to create the bucket, like this:

```
$ export BUCKET_NAME=s3://sensor-data-$USER
$ aws s3 mb ${BUCKET_NAME}
```

To set up the pipeline, go to the Date Pipeline page in the AWS console⁹ and select *Create new pipeline*, as you can see in the figure that follows.



When the pipeline creation page pops up, do the following:

- Give the pipeline any name you’d like.
- Under *Source*, select *Build using a template* and then the *Export DynamoDB table to S3* template.
- Under *Source DynamoDB table name*, specify the `SensorData` table, and under *Output s3 folder*, input the name of the S3 bucket that you created.
- Leave the *DynamoDB read throughput ratio* value as is and supply the AWS region (we’ve been using `us-east-1` thus far in these examples but yours may differ).
- Under *Run* in the *Schedule* section, select *on pipeline activation*. Disable logging in the next section and select *Default* under *Security/Access*.

Now click *Activate*. That will bring you back to the data pipeline console. The pipeline job will go through several phases, beginning with `WAITING`. Behind the scenes, Data Pipeline is actually spinning up, using, and finally shutting down an Elastic MapReduce cluster to perform the data transfer (think back to Day 3 of the HBase chapter when we spun up our own EMR cluster to run HBase). The job should finish within a few minutes. Once it’s done, every

9. <https://console.aws.amazon.com/datapipeline>

item in the SensorData table will be kept in a single file in S3, with each sensor reading as a JSON object on a separate line in the file.

Querying the Data Using Athena

Athena is an AWS service that enables you to query data files stored in S3 using plain SQL queries. We'll use Athena to get some aggregate metrics out of the data stored in our SensorData table. Go to the Athena console¹⁰ and click on *Query Editor*. This is the interface that you can use to create tables and run queries. Run this query to create a sensor_data table (substituting the appropriate name for the S3 bucket that you created).

```
CREATE EXTERNAL TABLE sensor_data (
  sensorid      struct<s:string>,
  currenttime   struct<n:bigint>,
  temperature   struct<n:float>
)
ROW FORMAT SERDE 'org.openx.data.jsonserde.JsonSerDe'
WITH serdeproperties ('paths'='SensorId,CurrentTime,Temperature')
LOCATION 's3://YOUR-S3-BUCKET/';
```

A few things to note here. First, we're using data structures called *structs* as columns. That's because the JSON object for each item in our DynamoDB table looks like this:

```
{
  "SensorId": {
    "S": "...",
  },
  "CurrentTime": {
    "N": "...",
  },
  "Temperature": {
    "N": "...",
  }
}
```

Structs will enable us to easily handle this nested JSON data. Second, the ROW FORMAT information simply specifies that we're querying files storing JSON as well as which JSON fields we're interested in (Sensorid and so on). Finally, the LOCATION points to our S3 bucket (don't forget the slash at the end).

The data from DynamoDB should all be in one file in a subfolder of the S3 bucket you created. The name of the subfolder is based on the current date and time. To see the name of that subfolder:

```
$ aws s3 ls ${BUCKET_NAME}/
```

10. <https://console.aws.amazon.com/athena>

Now, list the contents of that subfolder:

```
$ aws s3 ls ${BUCKET_NAME}/SUBFOLDER/
```

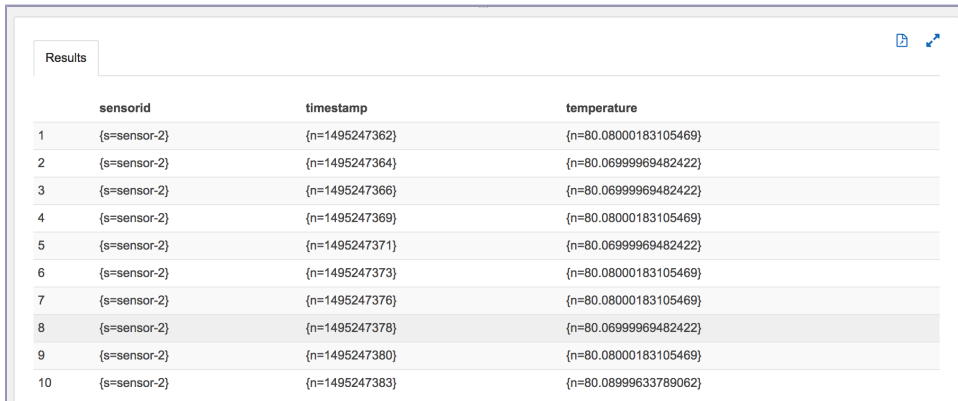
You’ll see that there are three files: a data file with a name like 5d2bf1ba-829e-402f-a9a4-2849b974be01, a manifest file, and a _SUCCESS file. Delete the latter two files, as they only contain metadata from Data Pipeline and will interfere with our Athena queries:

```
$ aws s3 rm ${BUCKET_NAME}/SUBFOLDER/manifest
```

```
$ aws s3 rm ${BUCKET_NAME}/SUBFOLDER/_SUCCESS
```

Now come back to the Athena console because you’re ready to run queries against the (now-sanitized) S3 folder. Let’s start with fetching 10 rows at random (see the figure that follows):

```
SELECT * FROM sensor_data LIMIT 10;
```



	sensorid	timestamp	temperature
1	{s=sensor-2}	{n=1495247362}	{n=80.08000183105469}
2	{s=sensor-2}	{n=1495247364}	{n=80.06999969482422}
3	{s=sensor-2}	{n=1495247366}	{n=80.06999969482422}
4	{s=sensor-2}	{n=1495247369}	{n=80.08000183105469}
5	{s=sensor-2}	{n=1495247371}	{n=80.06999969482422}
6	{s=sensor-2}	{n=1495247373}	{n=80.08000183105469}
7	{s=sensor-2}	{n=1495247376}	{n=80.08000183105469}
8	{s=sensor-2}	{n=1495247378}	{n=80.06999969482422}
9	{s=sensor-2}	{n=1495247380}	{n=80.08000183105469}
10	{s=sensor-2}	{n=1495247383}	{n=80.08999633789062}

Here are some other queries that you can run:

```
/* List the sensors for which you have data */
SELECT DISTINCT sensorid.s AS SensorId FROM sensor_data;
/* Get the number of readings from a specific sensor */
SELECT COUNT(*) AS NumberOfReadings FROM sensor_data
  WHERE sensorid.s = 'some-sensor-id';
/* Find the average temperature measured by a specific sensor */
SELECT AVG(temperature.n) AS AverageTemp FROM sensor_data
  WHERE sensorid.s = 'some-sensor-id';
/* Find the average temperature across all sensors */
SELECT AVG(temperature.n) AS AverageTempAllSensors FROM sensor_data;
/* Find the average temperature from a specific sensor */
SELECT AVG(temperature.n) AS AverageTemp FROM sensor_data
  WHERE sensorid.s = 'some-sensor-id';
/* Find the maximum temperature across all sensors */
SELECT MAX(temperature.n) AS MaxTemp FROM sensor_data;
/* Find the standard deviation for temperature across all sensors */
SELECT STDDEV(temperature.n) AS StdDev FROM sensor_data;
```

Production Ready? Not Quite

The system that we’ve built here—the Kinesis/Lambda/DynamoDB pipeline plus Athena querying layer—has served us well in exploring these services, but you wouldn’t want to use it as a production system. Why? Because it’s built on the assumption that there’s only one table.

In reality, data in DynamoDB is often spread across many tables, while querying that data involves knowing in which tables the right data lives (one *could* call this *tertiary indexing* but the authors are unaware of this usage in the wild!). For our sensor data use case, that may involve, for example, storing data from different days in different tables and automatically writing new sensor data to the correct table.

To make a setup like that work, you could auto-infer the table name from the current date in the Lambda function, like this:

```
var date = new Date().toJSON().slice(0,10).replace(/-/g, '-');
var tableName = `sensor-data-${date}`;
var item = {
  TableName: tableName,
  Item: {
    // item data
  }
}
```

The name of each table in this setup would be of the form `sensor-data-YYYY-MM-DD`. You could create DynamoDB tables for specific dates in advance, along with S3 buckets for holding JSON data and Athena external table definitions for those different buckets. This would, in turn, enable you to run queries across DynamoDB tables and thus to gather metrics across days. A query like this, for example, would find the average temperature for all sensors across two days’ worth of readings:

```
SELECT (
  (SELECT SUM(temperature.n) FROM sensor_data_05_01_2017) +
  (SELECT SUM(temperature.n) FROM sensor_data_05_02_2017)
) / (
  (SELECT COUNT(temperature.n) FROM sensor_data_05_01_2017) +
  (SELECT COUNT(temperature.n) FROM sensor_data_05_02_2017)
);
```

What a ride! We just combined five AWS services—DynamoDB, Kinesis, Lambda, DataPipeline, and Athena, plus some helper services like IAM—into a cohesive platform for sensor data.

Day 3 Homework

Find

1. We added a global secondary index to our SensorData table, which increased our disk space usage. Read some documentation¹¹ to see how much space GSIs use.
2. The Athena service enables you to use a broad palette of SQL commands for running queries against S3 buckets—but not *all* the commands you may be used to. Do some Googling and find out which commands are and aren't supported.

Do

1. Exercise 1 in the homework for Day 2 called for modifying the data pipeline to handle humidity readings from our mock sensors. Modify the sensor data input script from Day 3 so that each sensor reading contains a humidity reading. Make sure to use a random walk logic to keep those readings relatively un-volatile. Then create a new external table in Athena that can run queries against that new data model.
2. In the last section of Day 3, we stated that the data pipeline we built isn't quite production ready because we have only one table and that a better system would store data from different days in different tables. Begin building that system to the best of your ability, given what you've learned so far.

Wrap-Up

This chapter has been quite a ride! We explored not just the very interesting and powerful DynamoDB but also a broad swathe of AWS offerings, and we built a pretty darned scalable data pipeline for use in a far-away datacenter. DynamoDB presents us not just with a relatively ops-free experience but always with a data model that sits in a nice middle point between the granularity of SQL and the minimalism of most NoSQL data models.

DynamoDB's Strengths

The strengths of DynamoDB have been on display throughout this chapter. It enables you to skip all the installation and setup steps and get started building immediately. And if you build something small, there are few intrinsic limits to

11. <http://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GSI.html>

how big it can get (*if*, of course, you have the budget for it). DynamoDB can be tricky in places, especially when it comes to things like capacity provisioning, planning for indexes, and coming up with a data model that enables you to take advantage of DynamoDB's performance and scalability. But getting past the tricky parts unlocks a lot of possibilities.

Although DynamoDB requires you to give up some of the querying power of relational databases, its data and querying models provide some powerful constructs, such as indexes and range queries, that you won't find in a lot of NoSQL databases. And if you run into barriers, such as the inability to write SQL-style queries for tables, you may be able to use an external service to fill in the gaps, as we did on Day 3. Beyond all of this, DynamoDB's feature set continues to expand, as does the constellation of services surrounding DynamoDB in the AWS ecosystem.

DynamoDB's Weaknesses

The drawbacks presented by DynamoDB are directly reminiscent of other NoSQL databases. Despite some interesting querying and modeling constructs, you're probably better off using Postgres or another relational database unless you know for sure that you're dealing with a DynamoDB-shaped problem (or maybe an HBase-shaped problem, or Mongo, and so on). Even if you have a use case that truly calls for a big NoSQL database, you may have trouble getting your data model to mesh well with DynamoDB's partitioning system. And as always, there's the cost. Any database you use is going to cost money *somehow*, but DynamoDB and others don't always make financial sense. There may be times when it makes sense to run databases yourself, maybe on your own hardware and maybe in the cloud. Caveat emptor.

Parting Thoughts

Cloud databases aren't everyone's cup of tea. They require a lot of trust—trust in massive systems whose internals you can't see and trust in the future economic prospects of cloud providers themselves. These and other drawbacks are real, but there are some major upsides, as we hope to have shown in this chapter. We urge anybody with a strong interest in databases to give them a try. Even if you don't ever have a use case that requires a database like DynamoDB, we think that they're greatly empowering for developers and just downright fun. They turn your humble little laptop into a very powerful orchestration platform.

DynamoDB is a good place to start but we focused on it here mostly due to its influence and market share. There are *many* other options. If you want a DynamoDB-style experience with a SQL favor, have a look at Google's BigQuery or Amazon's Redshift. For globally distributed databases, try Google Cloud Spanner, CockroachDB, or FaunaDB. We reckon that the database landscape is starting to lean strongly in the direction of the cloud and the ops-free lifestyle. The next edition of this book may even feature several cloud databases.